SIMATS ENGINEERING

CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.

Student Name	**P V GIRIDHAR**	**Register Number**	**1972260022**
Department / Section	**CSE AI**	**Academic Year / Semester**	2025-2026
Faculty Name	**Dr. Krishna Kumar**	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	**2-11-2026**	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

	Task Title	**Max**	**Marks Awarded**	**Remarks**
1	Design the Agent and the Search Algorithms	5		
2	Identify the Basic Operation	2		
3	Derive Time and Space Complexity Functions	6		
4	Best, Average and Worst-Case Analysis	4		
5	Asymptotic Analysis	3		
6	Formulate the Recurrence Relation	3		
7	Solve the Recurrence Relation	4		
8	Implement and Test the Algorithms	4		
9	Record Experimental Performance	3		
10	Compare Theory with Experiments	4		
11	Recommend and Defend the Final Design	4		
12	Re-evaluate and Adapt the Design (Day 4)	8		
	TOTAL	**50**		

OVERALL REMARKS:

CSAI707 - Artificial Intelligence

CO1 - Assignment

P V Girisdhar
1972260022

Task-1 : (Agent & Algorithm Design)

PEAS	Robot
Performance	Correct delivery, least travel time, no collisions, low battery use.
Environment	Hospital corridors, junctions, lifts, ward doors, staff & trolleys.
Actuators	Wheels, lift / door interface, locked medicine tray.
Sensors	Lidar / Ultrasonic, Camera, wheel encoders, localisation, Barcode reader.

View of AI:

Acting rationally. The robot is a goal-based problem-solving agent. It picks the move sequence that reaches the ward at least travel time, which is what a rational agent does.

This is the classic AI problem solving agent setting: Newell Simon and Shaw's General Problem Solver (1957) searched state spaces.

Formulation	Definition
States / initial	Locations of the graph (N of them) / the pharmacy node.
Actions / transition	go(v) for each corridor (u,v); RESULT(u, go(v)) = v
Goal test / path cost.	u = requested ward g / Sum of corridor times.

Preprocessing:

1. BFS (iterative, queue): Expand shallowest first
queue = [s]; while queue: u = pop_front; for v in adj[u]: if v new: parent[v] = u; if v == g return path.

2. DFS (iterative, stack): Expand deepest first
stack = [s]; while stack: u = pop; if u == g return path; for v in adj[u]: if v new: parent[v] = u; push(v)

3. UCS (iterative, priority-queue)	Expand lowest cost g first; goal test when postponed.	Only one optional for travel time.
4. IDDFS (recursive)	Run depth-limited DFS with limit 0, 1, 2, ...	BFS-quality route with tiny memory;

TASK 2 - Basic Operation:

Algorithm	Where it occurs	How it grows with N
BFS/DFS	Inner loop over adj[u] for every expanded node.	Up to the no. of nodes seen, never more than N; in this hospital a const. fracti of N.
UCS	Same loop plus a heap push, O(log n) each.	Nodes with cost $< C^*$; at most $N+2E$ pushes.
IDDFS	Every recursive call of DLS, repeated in each iteration.	About b^d per final pass plus repeated shallower pass;

TASK

TASK3 - Time & Space Complexity:

BFS: Level i has b nodes. With goal test at generation, nodes generated $= 1+b+\ldots+b^d$
$\Rightarrow (b^{d+1}-1)/(b-1) = O(b^d)$. With an explored set each node is generated once & each considered examined twice, so also $\leq N$ and $O(N+E)$.

DFS: Tree form visits up to depth m.
graph form $\leq N$ generations, $O(N+E)$.
Space: $O(bm)$ in tree form these at most $N+2E$ heap pushes of $O(\log N)$ each: $O((N+E)\log N)$ time, $O(N)$ space.

IDDFS: Pass L generates $(b^{L+1}-1)/(b-1)$ nodes, so

$$T = \sum_{L=0}^{d} (b^{L+1}-1)/(b-1) = (b^{d+2}b-(d+1)(b-1))/(b-1)^2 b^d$$
$$\Rightarrow O(b^d)$$

Equivalently a node at depth j is regenerated in $(d-j+1)$ passes, so $T = \sum (d-j+1).b^j$.
Space is just the current path, $O(d)$.
Total $O(N+E)+O(b^d)$.

TASK-4 : Best, Average, Worst case.

Algorithm	Best case	Average case	Worst case.
BFS	Goal = start or a neighbor deg(s) nodes.	Goal at depth = log-b; the last layer has most nodes.	Goal in the deepest layer or unreachable; all N nodes.
DFS	Goal on the first branch fall.	Long wandering dive first.	All N nodes; it ignores cost, so cheap-edge detours do not matter.
UCS	Goal is the cheapest neighbor.	Goal at depth $d \approx 2.25 \cdot b^d$; duplicates are regenerat-ed.	Unreachable goal: it would have to enumerate every simple goal.

TASK 5 - Asymptotic Analysis :

- Growth : Here $d \simeq \log_b N$, so $b^d \approx N$ & all four are $\Theta(N)$ in time. Measured (log-log slopes of nodes generated from 1000 to 1,000,000.

BFS 0.96, DFS 1.04, UCS 0.99, IDDFS 0.93.

- memory limits first: A+ 1,000,000 locati
  BFS keeps ≈ 811,686 frontier entries &
  UCS ≈ 317,552. IDDFS only 1d.
- Caveat: Real buildings have long corridors,
  so d can pas exceed log N. Then $b^d >> N$
  IDDFS becomes hopeless while BFS/UCS
  stay bounded by N+E.

TASK-6 Recurrence Relation:

Recursive algorithm: depth-limited search
DLS(u,g,d). Reading its code: the test u=g
and the test l=0 cost constant time; then the
loop makes one recursive call DLS(v,g,d-1).

Component	In my algorithm.
Problem size	Remaining depth limit d
Subproblem size	d-1 (subtract by one)
No. of calls	b (one per child)
Work outside calls	constant c (goal test, limit test, path set lookup)
Base case	l=0; goal test, return CUTOFF, T(0)=c_0

TASK-7 Solving the Recurrence

The recurrence shrinks the problem by 1 rather than dividing the, so the master Theorem does not apply. It is linear with a constant term, so the repeated substitution works.

Meaning: Extra level of depth multiplies the work by $b=3$, while memory stays $\Theta(1)$: time-hungry memory-light-Iterative-deepening:

$I(d) = \sum_{L=0}^{d} (b^{L+1} - 1)/(b-1) = \underline{(b^{d+2} - b - (d+1)(b-1))}$

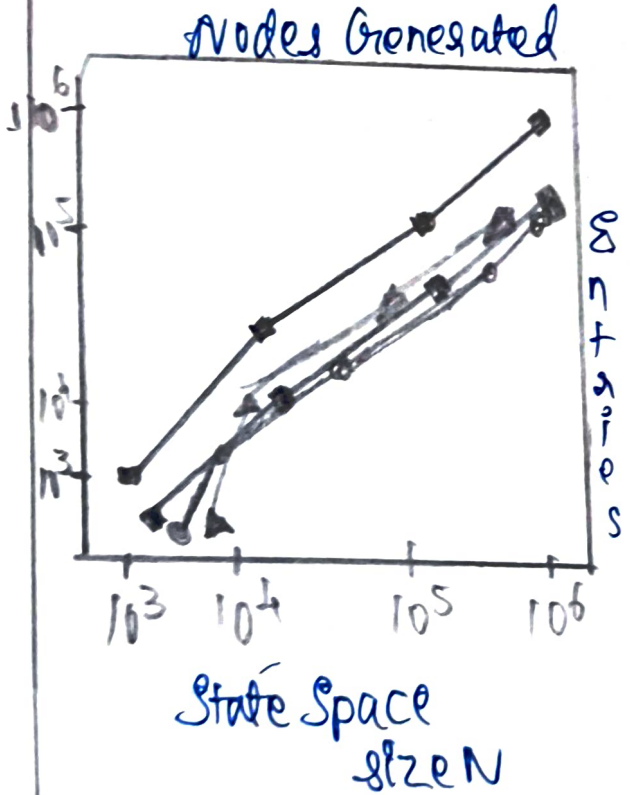
$\Rightarrow \Theta b^d$. For $b=3, d=12$ this $(b-1)^2$ is 1,195,735 calls against 797,161 for one pass of depth 12, only about 50% more (ratio $\to b/(b-1)$).

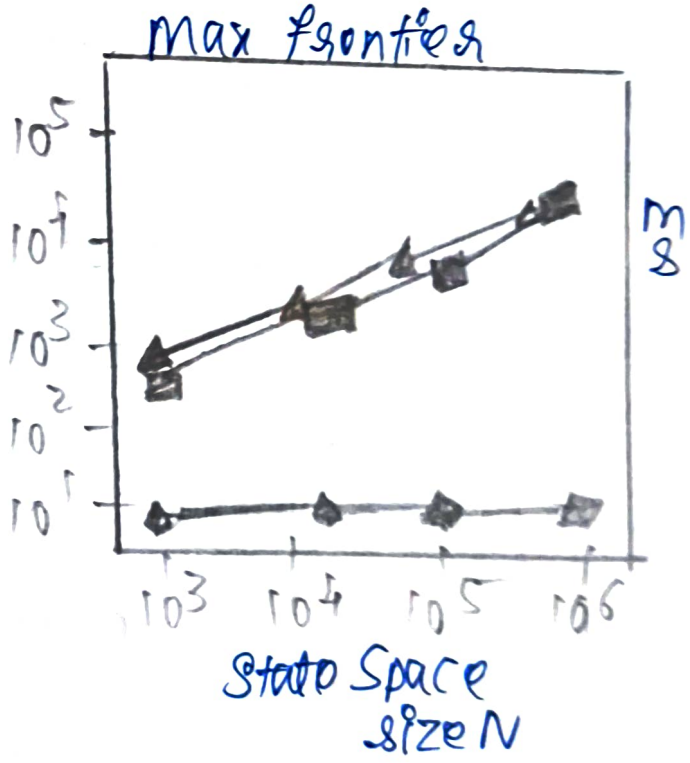
TASK-8 - Implementation & Testing:

Test	Expected	Actual
Reachable goal (open size).	valid route from all four; BFS/IDDFS fewest corridors.	All 144 of 144 runs valid. BFS fewest-corridors q/q at every size.

Start = goal	Route[s], 1 node generated.	All four return [s] with 1 generated at all sizes.
Unreachable goal.	BFS/DFS/UCS fail after exploring everything.	BFS, DFS generated exactly N.

TASK 9 - Experimental Performance:





TASK -10 - Theory vs Experiment:

- Agreement: Nodes generated grew linearly as predicted when $b^d \approx N$. Solution depth rose about 2 corridors per $\times 10$ in N (6.1 → 8.7 → 10.7 → 12.7) against $\log_3 10 = 2.1$. BFS frontier stayed ≈ 24%, 25%, 21%, 21% of N while IDDFS recursion depth grew like the depth.

- IDDFS formula: With $b = 3$ & each query's own depth, observed counts were 0.74, 0.54, 0.40, 0.52 of the task 7 prediction.

- Best Scalar: BFS, fewest nodes among the explored set methods & fastest. IDDFS wins on memory.

- Differences & reasons:

(1) Duplicates: BFS discards them, IDDFS regenerates them, so IDDFS used 4.2× BFS instead of the theoretical 1.5×.

(2) The goal is usually found before its layer is finished, so BFS generated 42–56% of N, not 100%.

TASK 11 - Final Recommendation:

* Objective is travel time: BFS minimizes corridors & was cost-optimal, in only 5, 4, 3, 1 of 9 queries per size. UCS in all 9. At 1M the mean cost was 38.9.

* Others rejected: DFS routes averaged 107,968 corridors for goals about 13 away.

* Efficiency: Theory $O((V+E)\log N)$ time & $O(N)$ memory; Observed 77%, 68%, 81%, 70% of N generated ≈ 1400 ms per query.

* Trade offs: About 5x slower than BFS & a frontier of ≈ 317,552 entries. If the robot cannot hold that, plan on the hospital server or use depth-capped IDDFS where d is small.

TASK 12 - DAY 4 Challenge:

A. Impact(2): The start is no longer one node but a set of K possible junctions, so the known-start & fully-observable assumptions fail.

B. Original Design: BFS/UCS cannot be used as they are (a state is no longer a location). They can run over belief states. There are upto 2^N beliefs, but an action never enlarges a belief.

C. Decision: MODIFY

Keep graph search with a frontier & explored set but changes the state to a belief state.
No sensor: BFS over belief finds one fixed move sequence that works for all k starts.